

Robust Model Predictive Control

Project Overview

Lorenzo Favaron
2128830

Luca Galeazzo
2158514

Lorenzo Trubian
2125512

September 30, 2026

1 Model Description

To validate the proposed Robust Model Predictive Controller, we evaluate its performance on a practical control scenario: the **Quadruple-Tank Process** [1]. This well-established multivariable benchmark system consists of four interconnected water tanks. The nonlinear system dynamics are linearized around a nominal operating point and discretized using a zero-order hold with a sampling time of $T_s = 1$ s, yielding the following discrete-time linear time-invariant (LTI) state-space representation:

$$\begin{aligned}x_{t+1} &= A x_t + B v_t + K u_t \\ y_t &= C x_t + D v_t\end{aligned}$$

where:

- $x_t \in \mathbb{R}^4$ represents the system states (water levels in the four tanks),
- $u_t \in \mathbb{R}^2$ denotes the control inputs (pump voltages),
- $y_t \in \mathbb{R}^2$ corresponds to the measured outputs (water levels in tanks 1 and 2),
- $v_t \in \mathbb{R}^6$ is the exogenous white noise signal.

The noise matrices, $B \in \mathbb{R}^{4 \times 6}$ and $D \in \mathbb{R}^{2 \times 6}$, are derived via the Cholesky factorization of a joint covariance matrix $\Sigma \in \mathbb{R}^{6 \times 6}$. This formulation ensures that the stacked matrix $[B^\top \ D^\top]^\top$ is square and full-rank. To facilitate robustness testing, the overall noise magnitude is scaled by a single tunable parameter applied to Σ . This allows for seamless transitions between low- and high-noise environments while preserving the underlying noise structure.

Note: The system matrices are generated using the `generate_quadruple_tank_model` function.

1.1 Simulation vs. Nominal Dynamic Models

To make the simulation more realistic, the model used by the controller — referred to as *nominal* — is intentionally set up to be different from the actual plant. This mismatch is introduced in two main ways.

The first is **deterministic model uncertainty**: the controller matrices are set to $A_{\text{nom}} = 0.95A$, $C_{\text{nom}} = 1.05C$, and $K_{\text{nom}} = 1.10K$ to simulate standard modelling errors.

The second part is **stochastic model uncertainty**. In the simulation, disturbances are generated using the Cholesky factors B and D of a joint covariance matrix Σ . However, the controller's nominal model uses a simpler diagonal structure:

$$B_{\text{nom}} = [\delta_{\text{process}} I_n \quad 0_{n \times p}], \quad D_{\text{nom}} = [0_{p \times n} \quad \delta_{\text{measurement}} I_p],$$

where $n = 4$ and $p = 2$ are the state and output dimensions, respectively. This approach was chosen because precise noise statistics are difficult to obtain in real-world applications. Instead of providing the controller with exact data, we only gave it a baseline, which serves as the center of an “uncertainty ball” for the robust filter. The tolerance parameter c then defines the size of this ball using relative entropy, allowing the filter to handle unexpected noise behaviors autonomously.

2 Code Logic

At a high level, the Robust MPC controller operates as illustrated in Figure 1. At each time step, a bank of Robust Kalman Filters - each parameterized by a different tolerance c_j - runs in parallel. A Tolerance Selector evaluates these filters, choosing the optimal tolerance c_t^* by balancing the MPC cost $\bar{J}_t(c_j)$ against a windowed prediction error. The selected state estimate (and, when operating in LFM mode, a time-varying prediction model) is then passed to the MPC Solver, which computes the optimal control input u_t^* .

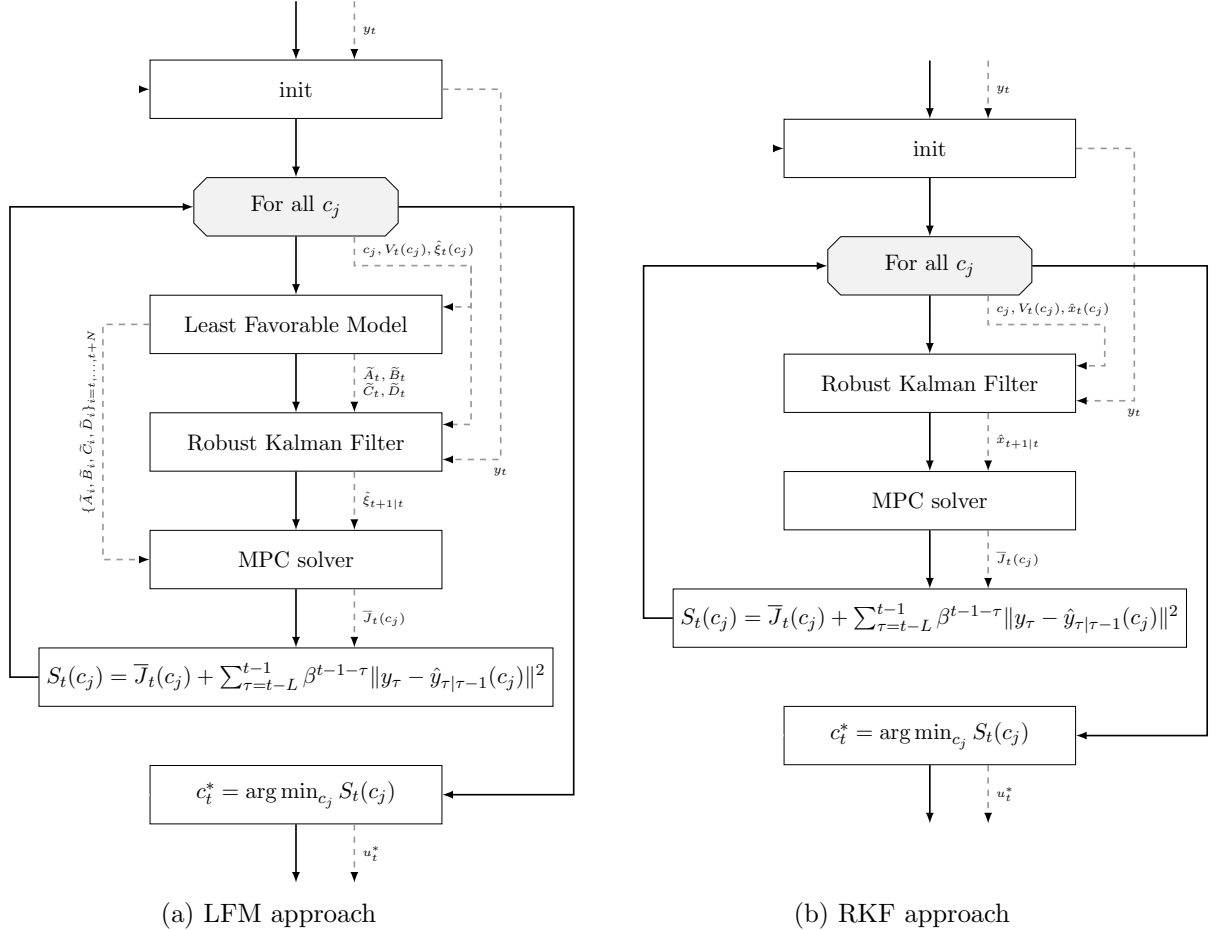


Figure 1: High-level block diagram of the proposed Robust MPC Controller

2.1 Major Components Explanation

Robust Kalman Filter bank. A discrete set of uncertainty radii $\mathcal{C} = \{c_1, c_2, \dots, c_M\}$ is defined *a priori*. If $c = 0$ is included in this set, it recovers the standard Kalman filter. At each time step t , a bank of M Robust Kalman Filters (RKFs) operates in parallel. Each filter j maintains its own least-favorable conditional covariance $V_t(c_j)$ and a one-step-ahead state estimate $\hat{x}_{t+1|t}(c_j)$. The filter gain $G_t(c_j)$ and updated covariance $V_{t+1}(c_j)$ are derived from a modified Riccati iteration. This iteration enforces a relative-entropy constraint of radius c_j around the nominal model and is solved using a Regula-Falsi root-finding procedure for the Lagrange multiplier $\lambda_t(c_j)$.

Own/Best prediction strategy. For each filter in the bank, the current prediction and covariance can be chosen between two natural alternatives: either the elements constructed in the previous step (Figure 2a and Figure 1), or those corresponding to the tolerance c_t^* selected by the MPCs in the previous step (Figure 2b).

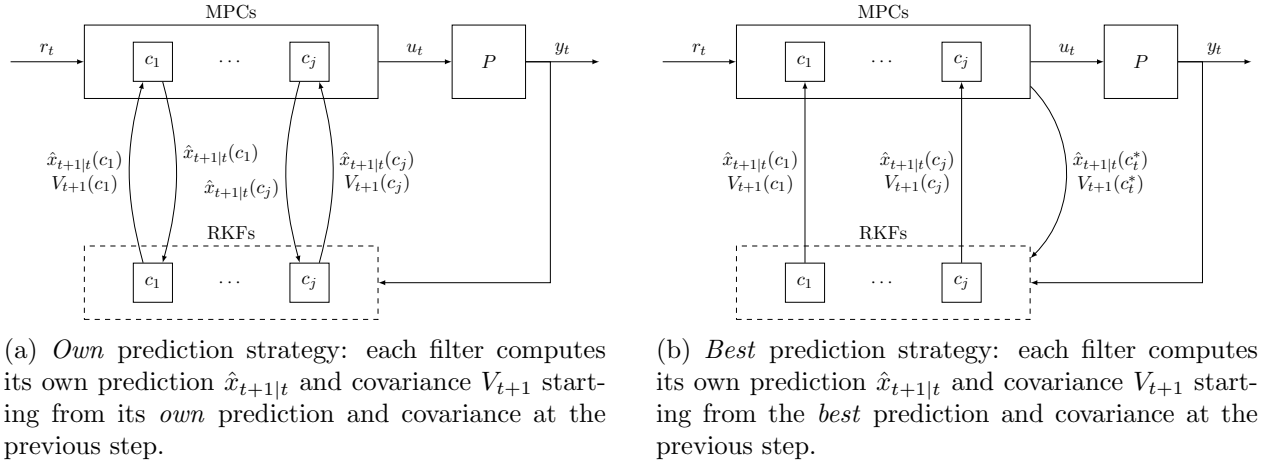


Figure 2: The dashed line emphasizes that no computations are performed across the RKF filters, but they share the measured output y_t , and eventually the initial estimate $\hat{x}_{t+1|t}(c_t^*)$.

Least Favorable Model (LFM) - optional mode. In the *LFM mode*, for each c_j the covariance $V_t(c_j)$ generates a time-varying prediction model for the MPC horizon. A forward Riccati sweep over $2N$ steps computes the sequence of filter gains, followed by a backward sweep that yields the matrices $\mathbf{H}$ and $\mathbf{L}$, which characterize the worst-case noise realization. The resulting Least Favorable Model features the $2n$ -dimensional extended state $\xi_t = (x_t^\top, e_t^\top)^\top$, where e_t tracks the estimation error. Its matrices $\tilde{A}_t$, $\tilde{B}_t$, $\tilde{C}_t$, and $\tilde{D}_t$ vary over the prediction horizon. See [2] as reference and Algorithm 5 for the pseudocode.

MPC Solver. For each candidate tolerance c_j , the MPC problem is formulated and solved as a Quadratic Programme (QP):

$$\begin{aligned} \bar{J}_t(c_j) &\triangleq \min_{\mathbf{u}_t} J(\hat{x}_{t+1|t}(c_j), \mathbf{u}_t) \\ &= \min_{\mathbf{u}_t} \left\{ \sum_{i=1}^{N-1} \|C_{\text{nom}} x_{t+i} - r_{t+i}\|_Q^2 + \sum_{i=0}^{N-1} \|u_{t+i}\|_R^2 + \|C_{\text{nom}} x_{t+N} - r_{t+N}\|_{P_f}^2 \right\}, \end{aligned} \quad (1)$$

This optimization is subject to input box constraints and the state evolution dictated by either the nominal or LFM dynamics. Because the equality constraints enforcing these dynamics are assembled as a sparse block system, the problem can be efficiently solved using solvers like OSQP or QUADPROG. Additionally, offset-free tracking is achieved by augmenting the nominal state with a fictitious constant-disturbance model. This compensates for a hardcoded mismatch introduced into the nominal dynamic model to simulate a modelling error.

Tolerance selection. After solving the M QPs, the controller identifies the optimal tolerance c_t^* by minimizing a composite score:

$$S_t(c_j) := \bar{J}_t(c_j) + \sum_{\tau=t-L}^{t-1} \beta^{t-1-\tau} \|y_\tau - \hat{y}_{\tau|\tau-1}(c_j)\|^2, \quad c_t^* = \arg \min_{c_j \in \mathcal{C}} S_t(c_j), \quad (2)$$

Here, $\hat{y}_{\tau|\tau-1}(c_j) := C\hat{x}_{\tau|\tau-1}(c_j)$ is the predicted output, $\bar{J}_t(c_j)$ is the MPC cost, L is the look-back window of past measurements, and $\beta \in (0, 1]$ is a forgetting factor. The first term rewards filters that yield a low control cost for the current prediction, while the second term penalizes those with inaccurate recent predictions. Finally, the optimal control input $u_t^* = u_t(c_t^*)$ is applied to the physical system. Before the next iteration begins, each filter's state estimate is corrected by incorporating the feedforward contribution of u_t^* .

3 Software Architecture

Figure 3 summarizes the organisation of the MATLAB implementation and the principal dependencies between its modules.

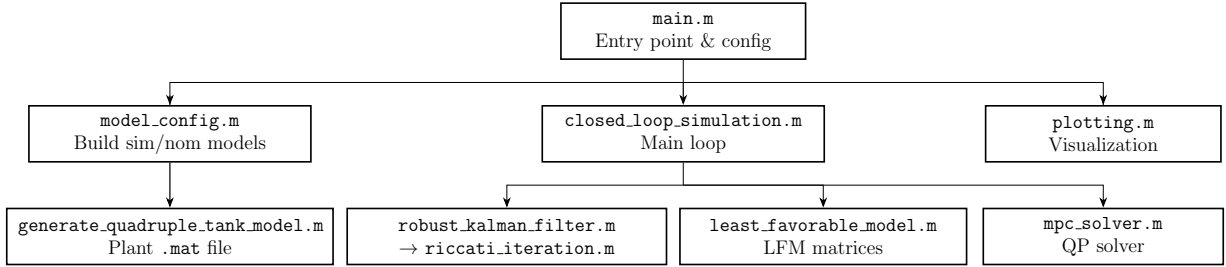
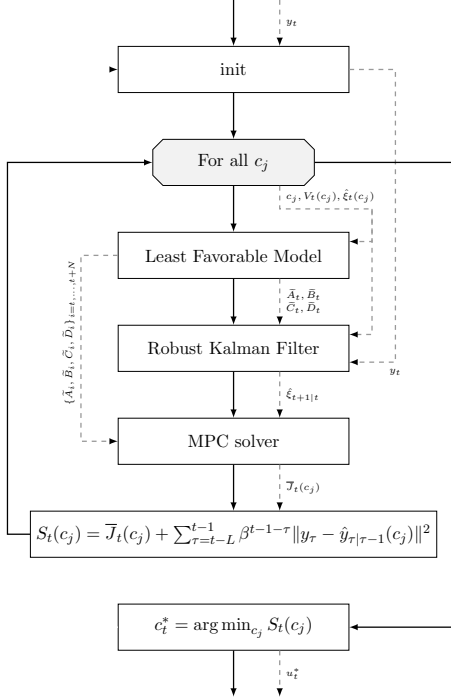


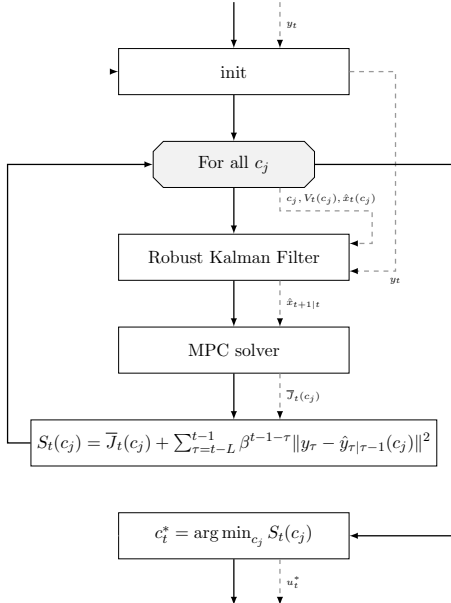
Figure 3: Software architecture of the MATLAB implementation.

Appendices

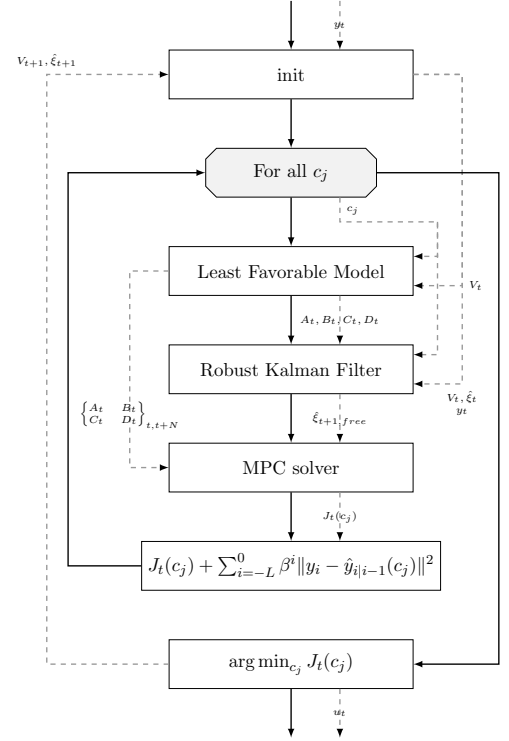
A Flowcharts



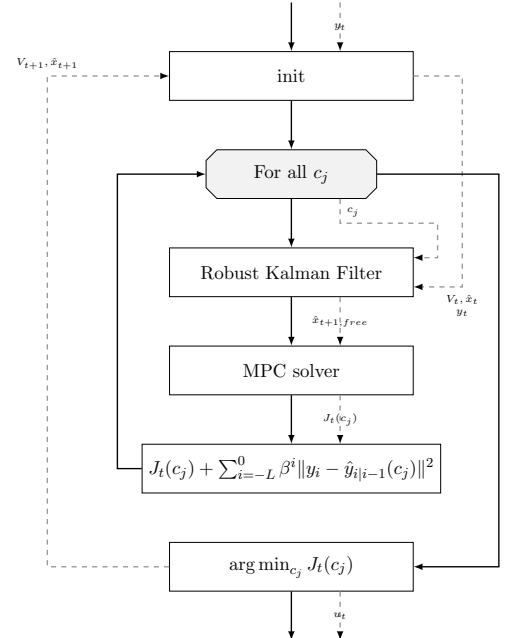
(a) LFM approach, own prediction strategy



(c) RKF approach, own prediction strategy



(b) LFM approach, best prediction strategy



(d) RKF approach, best prediction strategy. It is not different from using a single tolerance c .

Figure 4: LFM approach, best prediction strategy

B Pseudo-code

Algorithm 1 Robust MPC with tolerance selection at time t (LFM + best prediction).

Require: $\{c_j\}_j, \hat{\xi}_t, y_t, V_t, N, \beta, L$

Ensure: $u_t, \hat{\xi}_{t+1}, V_{t+1}$

```

1: for all  $c_j$  do
2:    $\triangleright$  Algorithm 5 to compute LFM in time interval  $[t, t + N]$ 
3:    $\{\tilde{A}_i, \tilde{B}_i, \tilde{C}_i, \tilde{D}_i\}_{i \in [t, t+N]} \leftarrow \text{LFM}(c, V_t, N)$ 
4:    $\triangleright$  Algorithm 4 applied using the best information of the previous step
5:    $\hat{\xi}_{t+1, \text{free}}(c_j), V_{t+1}(c_j) \leftarrow \text{RKf}(c_j, \hat{\xi}_t, y_t, V_t, \{\tilde{A}_t, \tilde{B}_t, \tilde{C}_t, \tilde{D}_t\})$ 
6:    $\triangleright$  MPC solver with LFM as nominal model
7:    $u_t(c_j), \hat{\xi}_{t+1}(c_j), J_t(c_j) \leftarrow \text{MPC}(\hat{\xi}_{t+1, \text{free}}(c_j), \{\tilde{A}_i, \tilde{B}_i, \tilde{C}_i, \tilde{D}_i\}_{i \in [t, t+N]}, N)$ 
8:    $\triangleright$  Add past prediction errors to the controller cost
9:    $J_t(c_j) \leftarrow J_t(c_j) + \sum_{i=-L}^0 \beta^i \|y_i - \hat{y}_{i|t-1}(c_j)\|^2$ 
10:  $\triangleright$  Optimization step, problem (??)
11:  $c_{\text{best}} \leftarrow \arg \min_{c_j} J_t(c_j)$ 
12:  $\triangleright$  Update prediction and covariance for the next step
13:  $\hat{\xi}_{t+1} \leftarrow \hat{\xi}_{t, \text{free}}(c_{\text{best}}) + \begin{bmatrix} K \\ 0 \end{bmatrix} u_t(c_{\text{best}})$ 
14:  $V_{t+1} \leftarrow V_{t+1}(c_{\text{best}})$ 
15:  $\triangleright$  Apply input
16:  $u_t \leftarrow u_t(c_{\text{best}})$ 

```

It is worth noting that step 13 of Algorithm 1 is actually redundant, since the solution z of the minimization problem (??) already includes the prediction of the next state $\hat{x}_{t+1}$. It is presented explicitly only to illustrate as clearly as possible how the next prediction is a combination of the Robust Kalman Filter and the Model Predictive Controller.

Algorithm 2 Robust MPC with tolerance selection at time t (LFM + own prediction).

Require: $\{c_j\}_j, y_t, N, \beta, L$

Ensure: u_t

```

1: for all  $c_j$  do
2:    $\triangleright$  Algorithm 5 to compute LFM in time interval  $[t, t + N]$ 
3:    $\{\tilde{A}_i, \tilde{B}_i, \tilde{C}_i, \tilde{D}_i\}_{i \in [t, t+N]} \leftarrow \text{LFM}(c, V_t(c_j), N)$ 
4:    $\triangleright$  Algorithm 4 applied using the information of the previous step of the filter associated to
      the same  $c_j$ 
5:    $\hat{\xi}_{t+1, \text{free}}(c_j), V_{t+1}(c_j) \leftarrow \text{RKf}(c_j, \hat{\xi}_t(c_j), y_t, V_t(c_j), \{\tilde{A}_t, \tilde{B}_t, \tilde{C}_t, \tilde{D}_t\})$ 
6:    $\triangleright$  MPC solver with LFM as nominal model
7:    $u_t(c_j), \hat{\xi}_{t+1}(c_j), J_t(c_j) \leftarrow \text{MPC}(\hat{\xi}_{t+1, \text{free}}(c_j), \{\tilde{A}_i, \tilde{B}_i, \tilde{C}_i, \tilde{D}_i\}_{i \in [t, t+N]}, N)$ 
8:    $\triangleright$  Add past prediction errors to the controller cost
9:    $J_t(c_j) \leftarrow J_t(c_j) + \sum_{i=-L}^0 \beta^i \|y_i - \hat{y}_{i|i-1}(c_j)\|^2$ 
10:  $\triangleright$  Optimization step, problem (??)
11:  $c_{\text{best}} \leftarrow \arg \min_{c_j} J_t(c_j)$ 
12:  $\triangleright$  Apply input
13:  $u_t \leftarrow u_t(c_{\text{best}})$ 

```

Algorithm 3 Robust MPC with tolerance selection at time t (RKf + own prediction).

Require: $\{c_j\}_j, y_t, N, \beta, L$

Ensure: u_t

```

1: for all  $c_j$  do
2:    $\triangleright$  Algorithm 4 applied using the information of the previous step of the filter associated to
      the same  $c_j$ 
3:    $\hat{\xi}_{t+1, \text{free}}(c_j), V_{t+1}(c_j) \leftarrow \text{RKf}(c_j, \hat{\xi}_t(c_j), y_t, V_t(c_j), \{A, B, C, D\})$ 
4:    $\triangleright$  MPC solver with nominal model
5:    $u_t(c_j), \hat{\xi}_{t+1}(c_j), J_t(c_j) \leftarrow \text{MPC}(\hat{\xi}_{t+1, \text{free}}(c_j), \{A, B, C, D\}, N)$ 
6:    $\triangleright$  Add past prediction errors to the controller cost
7:    $J_t(c_j) \leftarrow J_t(c_j) + \sum_{i=-L}^0 \beta^i \|y_i - \hat{y}_{i|i-1}(c_j)\|^2$ 
8:  $\triangleright$  Optimization step, problem (??)
9:  $c_{\text{best}} \leftarrow \arg \min_{c_j} J_t(c_j)$ 
10:  $\triangleright$  Apply input
11:  $u_t \leftarrow u_t(c_{\text{best}})$ 

```

Algorithm 4 Robust Kalman filter at time t .

Require: $c_t, \hat{x}_t, y_t, V_t$,**Ensure:** $\hat{x}_{t+1}, V_{t+1}, G_t, \lambda_t$

- 1: $G_t \leftarrow (A_t V_t C_t^\top + B_t D_t^\top)^\top (C_t V_t C_t^\top + D_t D_t^\top)^{-1}$
 - 2: $\hat{x}_{t+1} \leftarrow A_t \hat{x}_t + G_t (y_t - C_t \hat{x}_t)$
 - 3: $P_{t+1} \leftarrow (A_t - G_t C_t) V_t (A_t - G_t C_t)^\top + (B_t - G_t D_t)(B_t - G_t D_t)^\top$
 - 4: Find λ_t such that $c_t = \gamma_\tau(P_{t+1}, \lambda_t)$
 - 5: Compute
 $V_{t+1} \leftarrow (I_n - \lambda_t^{-1} P_{t+1})^{-1} P_{t+1}$
-

Given that it is evaluated repeatedly throughout the algorithm, and that subsequent computations depending on its value exhibit high sensitivity, both numerical stability and computational efficiency must be considered when determining λ_t (step 4 of Algorithm 4). In this work, we employ a variant of the regula falsi secant method, incorporating the modifications proposed by Anderson and Björk in [3] (a general pseudo-code for this family of algorithms can be found in [4]).

Algorithm 5 Least-Favorable Model for the time interval $[k, k + N]$.

Require: $c, V_k, N, N_f > N$ (default $N_f = 2N$)**Ensure:** $\tilde{A}_t, \tilde{B}_t, \tilde{C}_t, \tilde{D}_t$ for $t \in [k, k + N]$

- 1: $\triangleright$ *Forward Riccati iterations at least to large enough time N_f* $\triangleleft$
 - 2: **for** $t = 1, \dots, N_f$ **do**
 - 3: $\quad V_{t+1}, G_t, \lambda_t \leftarrow RKF(c, V_t)$ $\triangleright$ *Algorithm 4 without state-prediction (step 2)*
 - 4: $\triangleright$ *Backward sweep to compute LFM covariances* $\triangleleft$
 - 5: $W_{N_f}^{-1} \leftarrow \lambda_{N_f}^{-1} I_n$
 - 6: **for** $t = N_f - 1, \dots, 1$ **do**
 - 7: $\quad \triangleright$ *temporary matrices* $\triangleleft$
 - 8: $\quad A_{tmp} \leftarrow A_t - G_t C, \quad B_{tmp} \leftarrow B_t - G_t D$
 - 9: $\quad \triangleright$ *computation of K^{-1}, H, L* $\triangleleft$
 - 10: $\quad K_t^{-1} \leftarrow I_{n+p} - B_{tmp}^\top W_t^{-1} B_{tmp}$
 - 11: $\quad H_t \leftarrow K_t B_{tmp}^\top W_t^{-1} A_{tmp}$
 - 12: $\quad L_t \leftarrow \text{cholesky}(K^{-1})^{-1}$
 - 13: $\quad \triangleright$ *update values for next iteration* $\triangleleft$
 - 14: **if** $t = 1$ **then**
 - 15: $\quad$ exit cycle
 - 16: **else**
 - 17: $\quad \Omega_t^{-1} \leftarrow A_{tmp}^\top W_{t+1} A_{tmp} + H_t^\top K_t^{-1} H_t$
 - 18: $\quad W_t^{-1} \leftarrow \Omega_t^{-1} + \lambda_{t-1}^{-1} I_n$
 - 19: $\triangleright$ *Forward sweep to compute the LFM matrices to N* $\triangleleft$
 - 20: **for** $t = 1, \dots, N$ **do**
 - 21: $\quad A_{tmp} \leftarrow A_t - G_t C, \quad B_{tmp} \leftarrow B_t - G_t D$
 - 22: $\quad$ Compute the matrices of LFM:
 $\quad \tilde{A}_t \leftarrow \begin{bmatrix} A_t & B_t H_t \\ 0 & A_{tmp} + B_{tmp} H_t \end{bmatrix}, \quad \tilde{B}_t \leftarrow \begin{bmatrix} B_t \\ B_{tmp} \end{bmatrix} L_t, \quad \tilde{C}_t \leftarrow [C_t \quad D H_t], \quad \tilde{D}_t \leftarrow D_t L_t$
-

References

- [1] K. H. Johansson, “The quadruple-tank process: A multivariable laboratory process with an adjustable zero,” *IEEE Transactions on Control Systems Technology*, May 2000.
- [2] B. C. Levy and R. Nikoukhah, “Robust state space filtering under incremental model perturbations subject to a relative entropy tolerance,” *IEEE Trans. Autom. Control*, vol. 58, no. 3, pp. 682–695, Mar. 2013.
- [3] N. Anderson and Å. Björck, “A new high order method of regula falsi type for computing a root of an equation,” *BIT Numerical Mathematics*, vol. 13, no. 3, pp. 253–264, 1973.
- [4] S. M. L. Galdino, “A family of regula falsi root-finding methods,” 2011. [Online]. Available: <https://api.semanticscholar.org/CorpusID:15854811>